

Labyrinth Breach: A Reproducible Unity ML-Agents Benchmark for Asymmetric Multi-Agent Pursuit-Evasion in Dynamic Mazes

Muhammad Sikander Raheem, Usman Irshad Bhatti, Ahmed Jawed
25280017 25280099 25280040
MS AI, LUMS
AI641 – AI for Robotics

Abstract—We present Labyrinth Breach, a configurable Unity ML-Agents benchmark for asymmetric multi-agent pursuit-evasion in dynamic maze environments. Three Sentinel agents cooperatively pursue two Runner agents across procedurally generated layouts with shifting walls, exit zones, and partial observability. Agents observe through vector states, 360-degree raycasts, and last-known-position memory, and are trained with PPO under a tactical reward-shaping framework designed to reduce free-rider collapse, cluster convergence, and passive evasion. Beyond the environment itself, we contribute an auditable evaluation pipeline that routes raw Unity logs into run-scoped artifacts, computes outcome and coordination metrics, and supports multi-seed seen/unseen layout evaluation and ablation studies. Preliminary legacy runs suggest that curriculum training improves behavior as environments progress from open arenas to dynamic mazes, but publication-grade claims require the stricter multi-seed protocol described in this paper. The source code and trained models are publicly available.¹ A demonstration video of trained agents in the dynamic maze is available online.²

Index Terms—Multi-agent reinforcement learning, pursuit-evasion, dynamic maze, reward shaping, Unity ML-Agents, PPO

I. INTRODUCTION

Pursuit-evasion is a classical problem in robotics and multi-agent decision-making. One side attempts to capture while the other attempts to escape. The problem is a natural fit for reinforcement learning (RL) because the desired behavior is difficult to hand-code when the environment involves adversarial interaction, obstacles, partial information, and multiple cooperating agents.

The original course project requires RL-based multi-agent pursuit-evasion using Unity ML-Agents. We extend this requirement into a structured environment called *Labyrinth Breach*, where three Sentinels pursue two Runners inside a maze with exit zones and dynamic topology changes.

Simple chase environments do not capture the full difficulty of multi-agent coordination. In a richer maze setting, effective Sentinel behavior includes trapping, pincer movement, corridor denial, exit blocking, and route cutting. On the other side, effective Runner behavior includes distance management,

path selection, survival, and escape through exits. These requirements align the project with robotics-style navigation and coordination problems rather than only direct-motion pursuit.

Our contributions are as follows:

- A configurable asymmetric 3v2 pursuit-evasion environment with procedurally generated static and dynamic maze layouts.
- A tactical reward shaping framework (`v5_active_agents`) that targets free-rider collapse, clustering, and passive evasion through configurable shaping terms.
- A memory-augmented partial observation design combining 360-degree raycasts with last-known-position tracking.
- A three-phase curriculum learning pipeline (open arena → static maze → dynamic maze) with transfer learning between stages.
- A reproducible evaluation and evidence pipeline for multi-seed seen/unseen testing, coordination metrics, and ablations over memory, tactical rewards, dynamic walls, curriculum, and action assistance.

II. RELATED WORK

We organize the related work into four themes that directly support different aspects of our system design.

A. Pursuit-Evasion as a Multi-Agent Problem

Direct pursuit-evasion papers share the same capture-versus-escape structure as our task. In harder settings, useful behaviors go beyond direct motion and include route blocking, coordinated pressure, spatial entrapment, and strategic escape timing [5], [8], [10], [11], [15]. Recent work has also shown that coordinated pursuit strategies can emerge naturally from simple reward signals when the environment contains enough structure [16]. Our environment targets exactly these richer behaviors through maze topology and exit-based objectives.

B. Cooperative and Mixed MARL

Multi-agent reinforcement learning faces a non-stationarity challenge because all agents update simultaneously. MADDPG addressed this through centralized training with decentralized execution [2]. More recently, Yu et al. showed that PPO-based

¹<https://github.com/ahmedjawedaj/labyrinth-breach-marl>

²https://drive.google.com/file/d/1sxzKtpvVNh_UYYxchMTZzzZNKLDyh09w/view

methods serve as strong practical baselines in cooperative settings when tuned carefully [4]. We adopt PPO with parameter sharing as our primary training method because Unity ML-Agents supports it natively and provides a reproducible baseline; stronger centralized-credit baselines such as MA-POCA or MAPPO remain important comparison points for publication-grade evaluation.

C. Credit Assignment

When multiple agents cooperate under team-level objectives, one agent may contribute by blocking a route or creating pressure while another gets the final capture. Plain shared reward can be too coarse for learning such indirect contributions [3], [6], [7], [12]. We address this through tactical reward shaping that provides per-agent bonuses for coordination events such as pincer formations and corridor control.

D. Partial Observability

Pursuit-evasion becomes more meaningful when sensing is limited. With full global state, the task is easier and less realistic. Works on visibility-constrained pursuit-evasion show that line-of-sight, uncertainty, and memory fundamentally change the problem [8]–[10]. Our observation pipeline uses vector states combined with ray-based visibility and last-known-position memory rather than global state or camera images.

III. PROBLEM FORMULATION

We model the environment as a Partially Observable Stochastic Game (POSG) defined by the tuple $\langle N, \mathcal{S}, \{A_i\}, \{O_i\}, T, \{R_i\}, \gamma \rangle$ where $N=5$ agents (3 Sentinels, 2 Runners), $\mathcal{S}$ is the joint state space covering agent positions, velocities, maze configuration, and wall states, $A_i \in \mathbb{R}^2$ are continuous actions for each agent (horizontal and forward movement), O_i is the partial observation function (117 floats for Sentinels, 129 for Runners), T is the transition function governed by Unity physics, R_i is the team-conditioned reward function combining team-level terminal outcomes with role-specific shaping terms (Table II), and $\gamma=0.99$ is the discount factor.

The Sentinel team wins when both Runners are captured before timeout. The Runner side wins when at least one Runner reaches an exit zone or when the episode times out at 120 seconds (5,000 decision steps). Each episode also features dynamic wall shifts at configurable intervals, changing the maze topology during play.

This formulation supports the study of asymmetric team interaction, coordinated pursuit, tactical escape behavior, partial observability, dynamic environment changes, and generalization to unseen layouts.

IV. SYSTEM DESIGN

A. Environment Structure

The implementation includes four scenes of increasing complexity:

- **Open arena baseline:** flat arena with no walls.

TABLE I
OBSERVATION AND ACTION SPACE PER AGENT ROLE.

Component	Sentinel	Runner
Self state	10	10
Environment context	7	7
Ray perception (6 per ray)	84 (14 rays)	96 (16 rays)
Last-known-position memory	6	6
Opponent summary (2 nearest)	10	10
Total observation size	117	129
Action space	2 continuous	2 continuous

- **Static maze:** fixed procedurally generated corridors.
- **Dynamic maze:** walls shift at configurable intervals during episodes.
- **Unseen evaluation maze:** novel layouts not encountered during training.

The dynamic maze is the core environment. Walls are generated procedurally from a seed and can shift during the episode by raising or lowering wall pillars on a timer. This prevents memorization of fixed routes and creates trapping opportunities and escape adaptations in real time. Exit zones are placed explicitly for the Runner side, making evasion a goal-directed navigation task rather than pure survival.

B. Agent Architecture

The codebase separates agent logic, environment control, reward computation, sensing, and logging into distinct modules. `BaseAgent` handles movement, observation collection, and ML-Agents integration. `SentinelAgent` extends it with capture-radius logic and target tracking. `RunnerAgent` extends it with escape and captured-state management. The central `PursuitEvasionEnvController` manages episode lifecycle, capture detection, reward triggering, and terminal conditions.

C. Observation Design

The observation pipeline is state-based rather than camera-based. Table I shows the complete breakdown. Each agent receives a flat vector containing self-state (position, velocity, heading, alive flag), environment context (time remaining, nearest exit distance and direction, wall proximity, wall shift timer), 360-degree raycasts, last-known-position memory for targets that have left line of sight, and a summary of the two nearest opponents including their relative position, visibility, and threat pressure.

Sentinels cast 14 rays while Runners cast 16 rays. Each ray produces 6 floats: normalized hit distance and one-hot indicators for wall, exit, teammate, opponent, and other object types. This gives Runners wider sensing coverage because their survival depends on detecting threats from all directions.

D. Reward Design

The reward system uses the `v5_active_agents` configuration, which was iteratively developed to address three

TABLE II
REWARD WEIGHTS (V5_ACTIVE_AGENTS CONFIGURATION).

Sentinel Rewards	Weight
Team full capture (win)	+1.0
Timeout or escape (loss)	−1.0
Individual capture bonus	+0.25
Chase progress (per step)	+0.004
Chase regression	−0.003
Cluster penalty	−0.015
Idle penalty	−0.002
Wall loop penalty	−0.002
Exploration visit bonus	+0.001
Orbit stall penalty	−0.003
Runner Rewards	Weight
Team exit success (win)	+1.0
Both captured (loss)	−1.0
Tagged individually	−0.2
Survival step bonus	+0.0001
Exit approach bonus	+0.005
Exit approach regression	−0.002
Evade progress	+0.001
Threat approach penalty	−0.002
Wall loop penalty	−0.001
Exploration visit bonus	+0.001
Orbit stall penalty	−0.003

behavioral failure modes observed during training: (1) free-rider collapse, where one Sentinel learns to capture while others idle; (2) cluster convergence, where all Sentinels chase the same Runner in a single file; and (3) passive evasion, where Runners survive by standing still rather than actively seeking exits.

The reward function combines terminal win/loss signals with distance-based shaping for pursuit and evasion, anti-clustering penalties, idle penalties, exit-seeking shaping, and a tactical coordination layer that rewards events detected by a dedicated `TrapEventDetector` module. The tactical layer rewards pincer formations, corridor control, exit denial, and dead-end forcing. All weights are loaded from YAML configuration files at runtime. Table II lists the final reward weights.

E. Logging and Reproducibility

Each training and evaluation run produces four CSV artifacts under a run-specific directory: `episode_log.csv` (per-episode outcomes and totals), `agent_step_log.csv` (per-step positions, actions, and distances), `reward_audit.csv` (per-event reward attribution by agent, category, and magnitude), and `replay_events.csv` (timestamped capture, escape, wall-shift, and tactical events). This structure ties every reported metric back to raw per-step evidence.

V. TRAINING AND EVALUATION SETUP

A. Training Algorithm

We train both behaviors with PPO using parameter sharing within each team. All three Sentinels share a single policy network, and both Runners share a separate one. Table III

TABLE III
PPO HYPERPARAMETERS (SHARED BY BOTH BEHAVIORS).

Parameter	Static Maze	Dynamic Maze
Learning rate	3.0×10^{-4}	2.5×10^{-4}
Batch size	2048	2048
Buffer size	20480	40960
Entropy (β)	5.0×10^{-3}	4.0×10^{-3}
Clip range (ϵ)	0.2	0.2
GAE λ	0.95	0.95
Discount (γ)	0.99	0.99
Epochs per update	3	4
Hidden units	256	256
Hidden layers	2	2
Time horizon	128	160
Sentinel max steps	1,000,000	1,500,000
Runner max steps	1,000,000	1,000,000

TABLE IV
THREE-PHASE CURRICULUM PROGRESSION WITH TRANSFER LEARNING.

Phase	Scene	Steps	Transfer From
1. Open Arena	01_Baseline	500K	–
2. Static Maze	02_StaticMaze	1,000K	Phase 1
3. Dynamic Maze	03_DynamicMaze	1,500K	Phase 2
<i>Evaluation</i>	04_UnseenMaze	–	Phase 3

lists the hyperparameters. The learning rate follows a linear decay schedule over the training run. MA-POCA is supported in the configuration for future credit-assignment experiments but was not used in the results reported here.

B. Curriculum and Transfer Learning

Training follows a three-phase curriculum with transfer learning between stages, motivated by evidence that staged difficulty helps agents discover coordination skills that are hard to learn from sparse reward alone [13]. Each phase uses `-initialize-from` to bootstrap the policy network from the previous stage’s checkpoint, preserving learned pursuit and evasion skills while adapting to increased environmental complexity. Critically, the network architecture (256 hidden units, 2 layers) is kept identical across all phases to enable seamless weight transfer.

Table IV shows the progression. Phase 1 establishes basic pursuit-evasion in an open arena. Phase 2 introduces fixed maze walls, forcing agents to learn spatial navigation. Phase 3 adds dynamic wall shifts, requiring adaptation to changing topology.

C. Evaluation Protocol

We evaluate trained policies in inference mode using fixed ONNX checkpoints. The publication protocol uses three policy seeds (42, 101, 202), one seen dynamic-maze split, and at least five held-out unseen layout seeds (101, 202, 303, 404, 505). All evaluation runs use deterministic action selection and must produce run-scoped raw logs: `episode_log.csv`, `agent_step_log.csv`, `reward_audit.csv`, and `replay_events.csv`. A run

TABLE V
PRELIMINARY PER-PHASE TRAINING PERFORMANCE SUMMARY FROM
LEGACY ARTIFACTS.

Metric	Open+Static	Dynamic	Unseen
Episodes	3,228	2,492	30
Sentinel win rate	100.0%	66.8%	50.0%
Runner win rate	0.0%	33.2%	50.0%
Sentinel mean reward	+3.545	+0.680	+0.257
Runner mean reward	-2.407	-0.993	-0.181
Wall-loop penalty	-0.009	-0.364	-0.017
Cluster penalty	-0.013	-0.518	-0.035

is excluded from publication tables if any required log, meta-data file, or KPI summary is missing.

The evaluation protocol reports outcome metrics (Sentinel win rate, Runner win rate, escape rate, survival time, and capture times), coordination metrics (pincer episode rate, trap frequency, corridor-block events, exit-denial events, Sentinel spread, and Runner separation), path metrics (path efficiency and route-change proxies after wall shifts), and ablation deltas. Because the current environment includes optional action-assist blending in the controller, publication results must include an action-assist-off ablation before attributing coordination purely to learned policies.

VI. RESULTS

This section separates two evidence layers. First, we retain legacy course-project summaries as historical motivation. Second, we report a validated checkpoint diagnostic produced by the run-scoped pipeline in Section V-C. Neither layer is sufficient for final journal claims because independent policy-seed retraining and the full ablation matrix remain incomplete.

The preliminary Sentinel win rate follows a curriculum-driven trend: near-perfect capture in the open arena/static maze setting, reduced dominance in the dynamic maze, and a balanced outcome in a small unseen-layout evaluation. We treat this as motivation rather than proof: stronger claims require repeated evaluation across policy seeds and multiple held-out layouts with confidence intervals.

A. Validated Checkpoint Diagnostic

After correcting standalone scene, rule-config, and reward-config routing, we evaluated the fixed `v5_dynamicmaze` checkpoint pair across one seen layout and five held-out layout seeds. Three evaluation seeds were used, but these are not independent training seeds. Each matrix cell ran for 30 wall-clock seconds in deterministic inference mode. The audit accepted all 18 cells, all 108 required raw/KPI artifacts, and every episode-level scene/rule/reward provenance check.

The aggregate result remains Sentinel-favored, but held-out geometry matters: per-layout Sentinel win rate ranges from 33.3% on seed 505 to 88.2% on seed 202. This variation is evidence against a layout-invariant performance claim and motivates longer per-layout evaluation with independently trained policy seeds.

TABLE VI
VALIDATED FIXED-CHECKPOINT DIAGNOSTIC. CONFIDENCE INTERVALS
ARE 95% WILSON INTERVALS OVER COMPLETED EPISODES.

Metric	Seen	Five unseen layouts
Completed episodes	12	106
Sentinel wins	9 (75.0%)	73 (68.9%)
95% CI	[46.8, 91.1]%	[59.5, 76.9]%
Runner wins	3 (25.0%)	33 (31.1%)
Escape rate	0.0%	25.5%
Mean full capture time	13.33 s	21.04 s

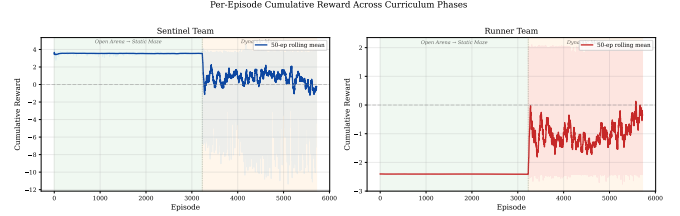


Fig. 1. Per-episode cumulative reward across curriculum phases. The curriculum transition (vertical line) shows the reward shift as maze complexity increases.

B. Reward Curves

Figure 1 shows the per-episode cumulative reward for both teams across all curriculum phases. The green-shaded region corresponds to the open arena and static maze phases, where the Sentinel reward is consistently high (+3.545). The orange-shaded region marks the transition to the dynamic maze, where the Sentinel mean reward drops sharply to +0.680 as the Runners learn to exploit wall shifts for cover.

Figure 2 decomposes the total reward into terminal, shaping, trap-aware, and penalty components. In the legacy run, terminal reward is the largest component, suggesting that shaping did not completely dominate win/loss learning. The publication pipeline verifies this claim by regenerating reward-breakdown tables from raw `reward_audit.csv` files for every run.

C. Win Rate Convergence

Figure 3 shows the 100-episode rolling win rate for both teams. In the static maze phase, the Sentinel win rate is near 100%. Upon entering the dynamic maze, the Runner win rate rises from near-zero toward a more balanced outcome. We avoid claiming a Nash equilibrium from these traces alone; the stronger interpretation is that dynamic topology makes evasion more viable and reduces Sentinel dominance.

D. Penalty Reduction as Evidence of Learning

Figure 4 tracks penalty signals for both teams across training. The Sentinel cluster penalty, introduced to prevent free-rider collapse, shows that agents initially bunched together but gradually learned to spread out. The Runner threat-approach penalty decreases over training, indicating that Runners learned to maintain safe distances from Sentinels rather than passively waiting.

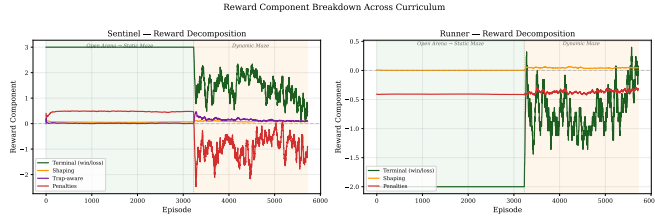


Fig. 2. Reward component decomposition. Terminal reward dominates; penalties increase with maze complexity.

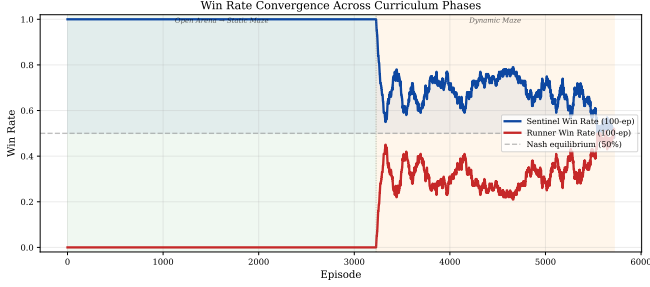


Fig. 3. Dual win rate trend. Sentinel (blue) and Runner (red) win rates become more balanced in the dynamic maze phase.

E. Shaping Signal Analysis

Figure 5 shows the distance-based shaping signals that guide pursuit and evasion. The Sentinel chase-progress signal increases steadily in the static maze as agents learn efficient pathfinding, then fluctuates in the dynamic maze as wall shifts disrupt previously learned routes. The Runner exit-approach signal activates strongly in the dynamic maze, suggesting that the `v5` exit-seeking shaping reduced the passive-evasion failure mode.

F. Per-Phase Summary

Figure 6 presents the per-phase comparison as bar charts. The progression motivates a curriculum-learning hypothesis, but final zero-shot generalization claims require the expanded unseen-layout protocol.

G. Held-Out Layout Transfer

The validated diagnostic confirms that the dynamic-maze checkpoint can execute in the held-out scene with seed-specific maze and reward configurations. Across 106 completed unseen-layout episodes, the aggregate Sentinel win rate is 68.9%, but the strong 33.3–88.2% per-layout spread prevents a robust zero-shot generalization claim. In addition, all cells use one trained checkpoint pair and short fixed-duration windows. The official experiment must therefore repeat the five-layout protocol using independently trained policy seeds and longer episode budgets.

VII. DISCUSSION

The results support three main findings.

Curriculum learning is a central hypothesis. The three-phase curriculum with transfer learning appears important for

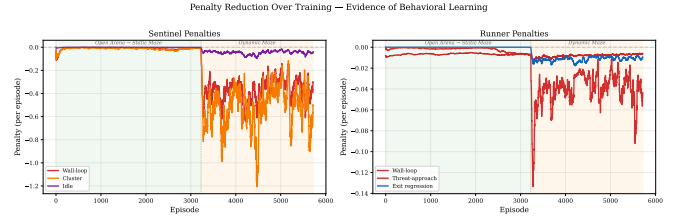


Fig. 4. Penalty reduction over training for Sentinel (left) and Runner (right). Decreasing penalties are consistent with reduced unproductive behaviours in the legacy run.

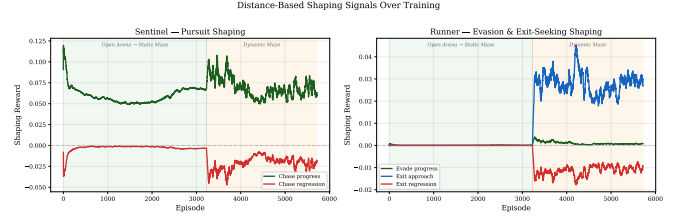


Fig. 5. Distance-based shaping signals. Sentinel chase progress (left) and Runner exit-seeking behaviour (right) across curriculum phases.

developing complex pursuit-evasion skills. However, this must be verified by a direct-dynamic-training ablation with the same network architecture and evaluation budget.

Custom reward shaping targets specific failure modes. The iterative development of the `v5_active_agents` reward configuration was driven by three observed failure modes:

- *Free-rider collapse:* In early training, one Sentinel learned to capture while others remained idle to collect the shared team reward. The `idle_penalty` ($-0.002/\text{step}$) and increased `individual_capture_bonus` ($+0.25$) were added to reduce this behavior.
- *Cluster convergence:* All three Sentinels would chase the same Runner in single file, making evasion trivial. The `cluster_penalty` (-0.015) encourages spatial separation; publication results must quantify this through Sentinel spread and pincer/corridor metrics.
- *Passive evasion:* Runners learned to survive by standing still, avoiding threat-approach penalties but never seeking exits. The `exit_approach_bonus` ($+0.005$) and `exit_approach_regression` (-0.002) were added to encourage goal-directed escape behaviour.

Generalization remains the key empirical test. The validated fixed-checkpoint diagnostic demonstrates executable transfer to five held-out layouts, but substantial layout sensitivity and the absence of independent policy seeds prevent a generalization claim. The official protocol tests whether performance and coordination metrics persist across both training seeds and held-out geometries.

Compared to MatrixWorld [9], which provides a grid-based pursuit-evasion benchmark, Labyrinth Breach operates in continuous 3D space with asymmetric teams, dynamic

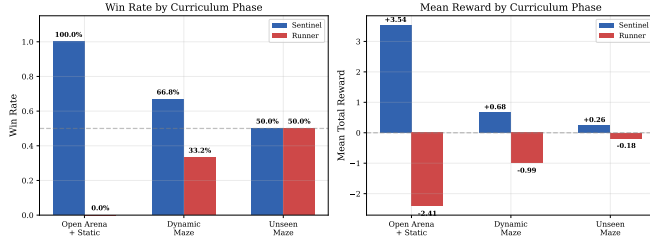


Fig. 6. Per-phase win rate (left) and mean reward (right). The progressive balancing of win rates motivates the curriculum-learning hypothesis tested by the publication protocol.

walls, and exit-based objectives. Compared to ViPER [10], which focuses on visibility-based pursuit, our environment adds team coordination pressure through the 3v2 asymmetry and tactical reward terms.

VIII. LIMITATIONS AND FUTURE WORK

Several limitations remain. First, the current study is simulation-only and does not include physical robot validation. Second, wall collision correctness under all dynamic shift configurations has not been exhaustively validated; rare edge cases during wall transitions could allow agents to clip through geometry. Third, the tactical reward shaping terms add complexity to the reward landscape and could be exploited in ways not yet detected. Fourth, the current architecture uses last-known-position memory rather than recurrent neural memory, which may limit performance in highly dynamic environments. Fifth, the controller includes optional action-assist blending; publication results must disclose it and include an assist-off ablation.

For future work, we plan to: (1) add LSTM memory and compare against the current last-known-position memory; (2) compare PPO with MA-POCA or MAPPO to study centralized credit assignment; (3) complete reward, memory, dynamic-wall, curriculum, and action-assist ablations; and (4) extend the benchmark toward hardware or high-fidelity robot simulators.

IX. CONCLUSION

We presented Labyrinth Breach, a configurable Unity ML-Agents benchmark for asymmetric multi-agent pursuit-evasion in dynamic mazes. The environment combines dynamic topology, partial observability, exit-directed evasion, and tactical reward shaping in a reproducible 3v2 setting. Preliminary legacy results motivate curriculum learning and reward-shaping hypotheses, but final publication claims require the strict multi-seed evaluation and ablation protocol introduced here. The environment, training configurations, logging pipeline, and trained model checkpoints are publicly available for reproduction and extension.

ACKNOWLEDGMENT

We acknowledge the AI641 course project brief for defining the original pursuit-evasion direction and motivating the Unity ML-Agents based implementation.

REFERENCES

- [1] AI641 – AI for Robotics, “Projects,” course handout, Project 2: Reinforcement Learning-based multi-agent pursuit-evasion.
- [2] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, “Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments,” in *Advances in Neural Information Processing Systems*, 2017.
- [3] T. Rashid, M. Samvelyan, C. S. de Witt, G. Farquhar, J. Foerster, and S. Whiteson, “QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning,” in *Proceedings of the 35th International Conference on Machine Learning Workshop*, 2018.
- [4] C. Yu et al., “The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games,” in *Advances in Neural Information Processing Systems*, 2022.
- [5] T. Verma, F. M. Zegers, and P. P. Khargonekar, “Multi-Agent Pursuer-Evader Learning using Situation Report,” arXiv preprint arXiv:1910.07780, 2019.
- [6] J. Wang, Y. Ren, T. Yu, H. Zhang, and C. Zhang, “Learning Implicit Credit Assignment for Cooperative Multi-Agent Reinforcement Learning,” in *Advances in Neural Information Processing Systems*, 2020.
- [7] C. Chen, H. Wang, Y. Hu, and C. Zhang, “Learning Explicit Credit Assignment for Cooperative Multi-Agent Reinforcement Learning via Polarization Policy Gradient,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.
- [8] O. Gonultas and V. Isler, “Learning to Play Pursuit-Evasion with Dynamic and Sensor Constraints,” arXiv preprint arXiv:2405.05372, 2024.
- [9] L. Sun, Y.-C. Chang, C. Lyu, C.-T. Lin, and Y. Shi, “MatrixWorld: A Pursuit-Evasion Platform for Safe Multi-Agent Coordination and Autocurricula,” arXiv preprint arXiv:2307.14854, 2023.
- [10] Y. Wang, Y. Cao, J. Chiun, S. Koley, M. Pham, and G. A. Sartoretti, “ViPER: Visibility-based Pursuit-Evasion via Reinforcement Learning,” in *Proceedings of the 8th Conference on Robot Learning*, PMLR, vol. 270, pp. 4188–4200, 2025.
- [11] F. Yan, J. Jiang, K. Di, Y. Jiang, and Z. Hao, “Multiagent Pursuit-Evasion Problem with the Pursuers Moving at Uncertain Speeds,” *Journal of Intelligent & Robotic Systems*, vol. 95, no. 1, pp. 119–135, 2019.
- [12] J. Foerster, G. Farquhar, T. Afouras, V. Nardelli, and S. Whiteson, “Counterfactual Multi-Agent Policy Gradients,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- [13] Y. Zhang et al., “Curriculum Learning for Cooperation in Multi-Agent Reinforcement Learning,” in *International Conference on Learning Representations (ICLR) Workshop*, 2023.
- [14] W. Li et al., “Team and Individual Rewards for Cooperative Multi-Agent Reinforcement Learning,” arXiv preprint arXiv:2109.10860, 2021.
- [15] Y. Liu et al., “Multi-UAV Pursuit-Evasion Gaming Based on PSO-M3DDPG Schemes,” *Complex & Intelligent Systems*, vol. 10, pp. 5221–5239, 2024.
- [16] H. Haeri, K. Shi, and J. Jeong, “Emergent Behaviors in Multi-Agent Target Acquisition,” arXiv preprint arXiv:2212.07891, 2022.